

Hebrew Rap Understanding Benchmark: Evaluating Large Language Models on Subcultural Israeli Idioms, Wordplay, and Metaphors

Raz Diamond

Department of Computer Science, Reichman University (RUNI)
raz.diamond@post.runi.ac.il

Submitted as final project report for the NLP course, Spring Semester 2026

Abstract

While modern Large Language Models (LLMs) are getting very good at understanding standard Hebrew, their ability to understand subcultural Israeli slang, wordplay, and rap metaphors is not well tested. Standard Hebrew benchmarks usually focus on formal text like news articles and Wikipedia. To solve this, I built the **Hebrew Rap Understanding Benchmark**, a manually curated dataset of 123 gold-standard multiple-choice and open-ended questions. The data was extracted from crowdsourced annotations on Genius.com. I evaluated 24 different models using three methods: multiple-choice accuracy, embedding cosine similarity, and an LLM-as-a-Judge approach. The results show a clear gap: top-tier general models (like Gemini) achieved up to 95% accuracy and an 83.35% final score, while specialized local Hebrew models (like DictaLM and Hebatron) struggled in open-ended explanations (scoring between 23% and 32%), often translating slang literally. Finally, I deployed a human-evaluation quiz on Wayground to establish a human baseline for this subcultural domain. All my code, datasets, and results are publicly available on GitHub.

1 Introduction

In recent years, Natural Language Processing (NLP) for Hebrew has made great progress, especially with the release of local models like DictaLM [1]. However, the existing evaluation benchmarks for Hebrew focus mostly on formal, grammatical, and standard text. Because of this, we don't really know how well these models understand informal street language, figurative speech, and modern slang.

Israeli hip-hop is a great source of modern Hebrew slang. It mixes military jargon, Arabic loanwords, street metaphors, and clever wordplay. Explaining a line of Hebrew rap requires understanding the cultural context, not just looking up words in a dictionary. For example, the phrase “שורף את המיקרופון” literally means “*burning the microphone*”, but in context, it means performing with high energy or showing great lyrical skill.

To test how well current LLMs handle this kind of language, I created the **Hebrew Rap Understanding Benchmark**. My main contributions in this project are:

1. **Dataset Curation:** I built a high-quality dataset of 123 evaluation items by scraping annotations from Genius.com, formatting them into multiple-choice questions and free-text prompts.
2. **Evaluation Pipeline:** I developed an automated pipeline that tests models using multiple-choice accuracy, semantic vector similarity, and the LLM-as-a-Judge method [2].
3. **Model Benchmarking:** I evaluated 24 different models, ranging from local open-weights models (DictaLM, Hebatron, Llama) to top-tier commercial APIs (Gemini, GPT-5).

4. **Human Baseline Integration:** I created an interactive web-based quiz exported from the dataset to compare LLM performance against real human comprehension.

1.1 Related Work

Evaluating how well models understand idioms, slang, and figurative language is a known challenge in NLP. While there are popular frameworks for evaluating English models (such as MT-Bench [2]), Hebrew NLP evaluation is still mostly focused on standard tasks like question answering and summarization (e.g., the datasets used to evaluate DictaLM [1]). Before this project, there was no standard benchmark specifically designed to test how Hebrew LLMs handle musical metaphors and subcultural slang.

2 Methodology

Building a clean, high-quality dataset from crowdsourced websites requires careful filtering and manual work. I used a Jupyter notebook (`01_dataset_pipeline.ipynb`) to process the data in several stages.

2.1 Data Collection and Filtering

1. **Data Harvesting:** I queried the website [Genius.com](https://www.genius.com) using specific genre tags (like "israeli-hip-hop" and "israeli-rap") to extract song lyrics and their crowdsourced annotations.
2. **Parsing:** I flattened the raw JSON data to extract the specific slang phrases along with their surrounding context stanzas and user explanations.
3. **Automated Filtering:** To eliminate noise before manual review, I applied a strict set of programmatic rules. I automatically removed structural song tags (e.g., phrases enclosed in brackets like [Chorus]), enforced language detection to ensure the phrase contained actual Hebrew characters, and applied community quality gates by requiring the annotation to have a positive net vote score and an "accepted" status from Genius moderators. Finally, to ensure dataset diversity, I capped the maximum number of candidates at 8 phrases per artist so the benchmark would not be dominated by a single prolific rapper.
4. **Human Curation:** I built an interactive widget in my Jupyter notebook to manually review the top 300 candidates that survived the automated filter. To avoid subjective bias, I graded each phrase using a strict taxonomy. Phrases were kept if they demonstrated lexical ambiguity/wordplay, required external cultural knowledge, or featured localized idiomatic slang. Conversely, phrases were discarded if they relied on private artist lore, were trivially easy (low semantic friction), or featured flawed annotations. This rigorous process yielded exactly 123 "gold" instances.

2.2 Prompt Engineering & Distractor Generation

To turn the curated data into a multiple-choice exam, I needed plausible wrong answers (distractors). I used `gemini-3.1-pro-preview` to generate three distractors for each correct explanation. The prompt was carefully written to ensure the distractors were culturally believable and matched the length and tone of the real answer. The system prompt used was:

You are an elite expert in Israeli hip-hop culture, Hebrew slang, and rigorous NLP benchmark design. I will provide a Hebrew rap lyric, its artist, and the crowdsourced explanation of its meaning.

Your task:

1. Summarize/rephrase the TRUE explanation into one clear, concise sentence (max 25 words) that sounds natural and professional.
2. Generate 3 plausible, highly convincing FALSE explanations (distractors) designed to genuinely trick an expert or an AI.
 - Each distractor must represent a believable alternative theory that a casual listener might mistake for the truth.
 - Avoid silly, absurd, or obviously fake options...

2.3 Example Benchmark Item

Table 1 shows an example of a finalized question from the benchmark dataset after the pipeline processing.

Table 1: Example item from 05_gold_benchmark.json.

Field	Content
Artist & Song	Tuna – “Rockstar”
Context Stanza	עושה את זה שנים, לא מזיע בחזרה / שורף ת'כמה ואז הולך למטרה...
Target Phrase	שורף ת'כמה
Gold Explanation	הופעה בעוצמה גבוהה, שליטה מלאה בקהל והפגנת כישרון יוצא דופן על הבמה.
Options (MC)	A: גרימת נזק פיזי לצידוד ההגברה במהלך ההופעה. B (Correct): הופעה בעוצמה גבוהה, שליטה מלאה בקהל והפגנת כישרון יוצא דופן. C: שימוש באפקטים של פירוטכניקה ואש במהלך השיר. D: ביטול ההופעה ברגע האחרון בשל חילוקי דעות עם המפיק.

2.4 Experimental Settings

To evaluate the models, I wrote an automated script (02_run_inference.py) to query the models, and a notebook (03_evaluate_benchmark.ipynb) to calculate the scores. The models were tested on three metrics:

1. **Multiple-Choice (30% Weight):** The model is given the stanza, phrase, and four options, and must return the correct letter.
2. **Cosine Similarity (20% Weight):** The model generates a free-text explanation of the phrase. I used `gemini-embedding-2-preview` to calculate the cosine similarity between the model’s answer and the real human explanation.
3. **LLM-as-a-Judge (50% Weight):** The generated explanations were sent to `gemini-3.8-flash`, which acted as a judge to grade the response on a scale of 1 to 5 based on accuracy and understanding of the slang [2].

3 Experimental Results

I evaluated 24 models in total. The full results are shown in Table 2.

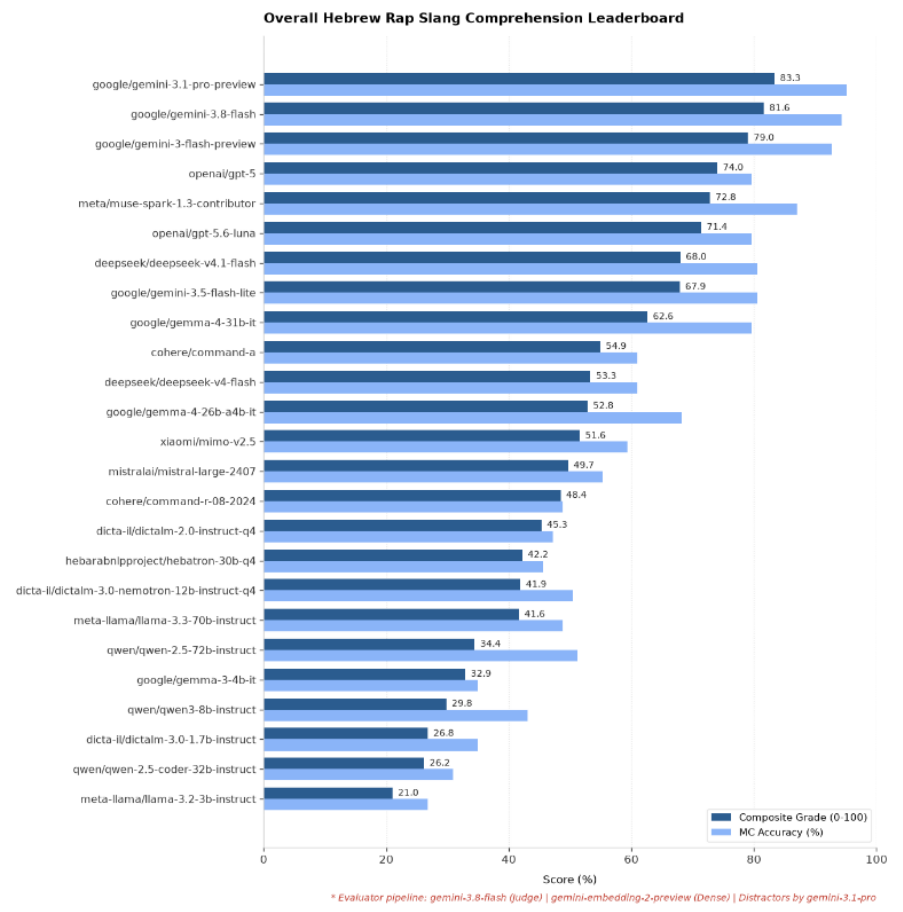


Figure 2: Overall Comprehension Leaderboard bar plot

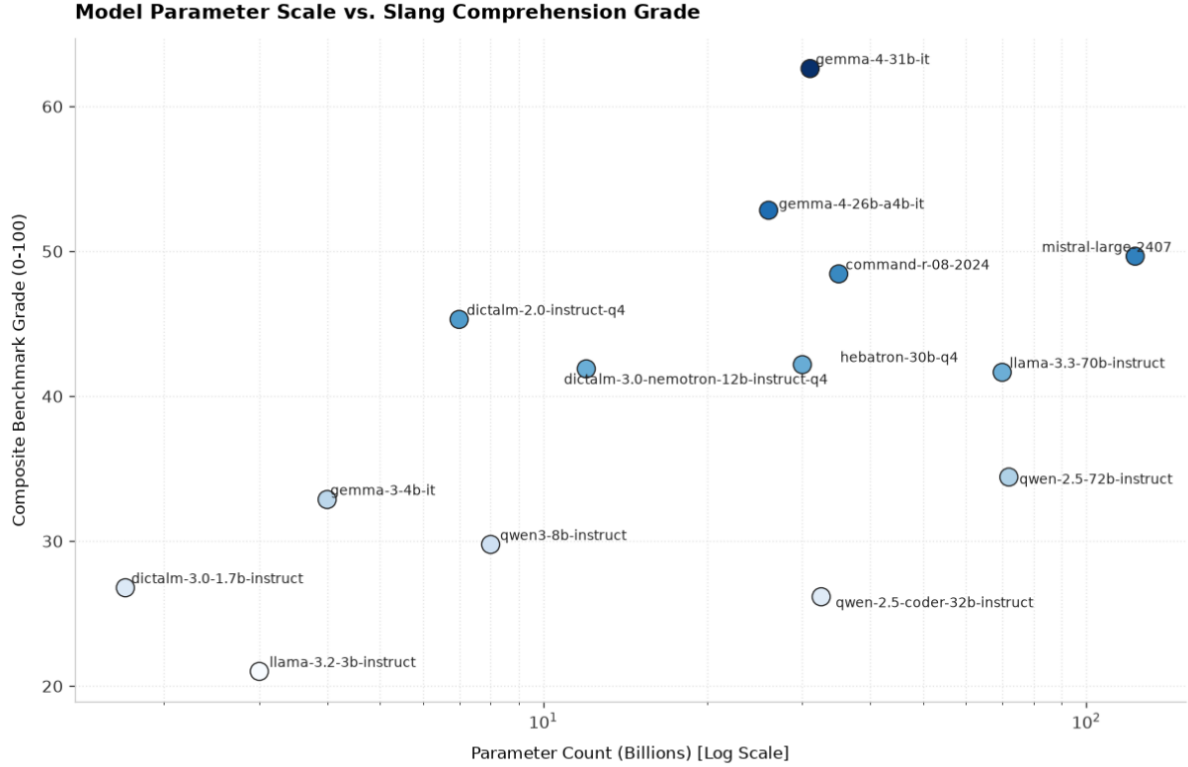


Figure 3: Log-Scale Parameter Count vs. Performance Graph (disclosed params models only)

3.1 Note on Excluded Models and Embeddings

It is worth noting that while the benchmark includes 24 diverse models, it does not currently feature some of the most expensive, heavy-weight proprietary models (such as Grok 4.7 or Claude Opus 5.5). Because the evaluation pipeline relies on API calls for both inference and the LLM-as-a-Judge grading step, I intentionally prioritized models that were cost-effective to inference for this initial study. However, the evaluation framework and automated scripts were built to be highly modular, meaning future researchers can effortlessly plug in and evaluate additional state-of-the-art models as needed.

Regarding the local models, I attempted to evaluate the SOTA model DictaLM 3.0 24B, but unfortunately, only a "thinking" (reasoning) version of the model was available rather than a standard instruct version. This caused infinite loops and extremely slow inference times during the open-ended tests, forcing me to exclude it from the final leaderboard.

Additionally, looking at the results, the cosine similarity embedding scores are clustered very closely together across all models (mostly between 70% and 78%). This happens because simply outputting text in Hebrew that repeats some of the keywords from the prompt is enough to achieve a decent similarity score, even if the semantic nuance is incorrect. I utilized Gemini's embedding model because the explanations can be lengthy and required a robust generalist model. However, it is highly probable that researching and integrating a dedicated, fine-tuned Hebrew embedding model could create a wider, more accurate variance in these scores in future iterations.

4 Discussion & Error Analysis

Working on this project revealed several interesting insights about how LLMs handle informal Hebrew.

4.1 The Literal Translation Fallacy

The most common mistake weaker models made was taking slang phrases literally. Instead of treating the phrase as one cultural concept, they broke it down word-by-word.

Table 3 demonstrates this phenomenon perfectly. When asked to interpret the phrase “איבדו” in the context of the war (where residents were evacuated from the literal North of Israel, while simultaneously feeling like they were going crazy), advanced models caught the double meaning. Local models entirely missed it.

Table 3: Qualitative Error Analysis: Literal vs. Figurative Interpretation.

Target Phrase	אמא שלך לא חוזרת לקריית שמונה, אשכרה איבדו תלפון
Ground Truth	משחק מילים: עזיבת קריית שמונה (הצפון הפיזי) במקביל לביטוי הסלנג "לאבד את הצפון" (להשתגע מרוב לחץ).
Gemini 3.1 Pro (Correct)	הדובר משתמש במשחק מילים כפול: אובדן הבית הפיזי בצפון הארץ עקב המלחמה, יחד עם הביטוי שמשמעותו אובדן שפיות.
DictaLM 2.0 (Literal Failure)	הדובר מתלונן על כך שהאמא לא רוצה לחזור לעיר קריית שמונה, ומציין שהם הלכו לאיבוד ולא מוצאים את הדרך צפונה.

4.2 Struggles of Local Hebrew Models

I expected models like DictaLM and Hebatron to perform very well because they are trained specifically on Hebrew. While they did okay on the multiple-choice section, they performed poorly on the open-ended explanations. This is likely because their training data is mostly formal text (like news and religious texts), so they simply do not know modern street slang.

4.3 Human Evaluation Baseline Preparation

To put the LLM results into perspective, I wanted to establish a framework for how humans perform on this exact same task. I wrote an export script (`scripts/export_wayground.py`) that takes a randomized 30-question subset of the benchmark and automatically formats it for the gamified quiz platform Wayground. While large-scale human testing is outside the scope of this initial report, the deployment of this tool creates a standardized, engaging environment to compare human intuition against the generated distractors in future studies.

4.4 Potential Benchmark Bias

It is important to mention a limitation of this project. Because I used `gemini-3.1-pro-preview` to generate the distractors for the multiple-choice questions, and I used `gemini-3.8-flash` as the judge for the open-ended answers, there is a strong inherent bias favoring Google’s Gemini models. This is a known phenomenon where an LLM prefers the style and syntax of its own model family, which partly explains why Gemini models completely dominate the top 3 spots on the leaderboard.

5 Code

The complete source code, evaluation datasets, and benchmarking scripts (including a link to the interactive human quiz on Wayground) are available in my public GitHub repository:

<https://github.com/razdiamond/hebrew-rap-slang-benchmark>

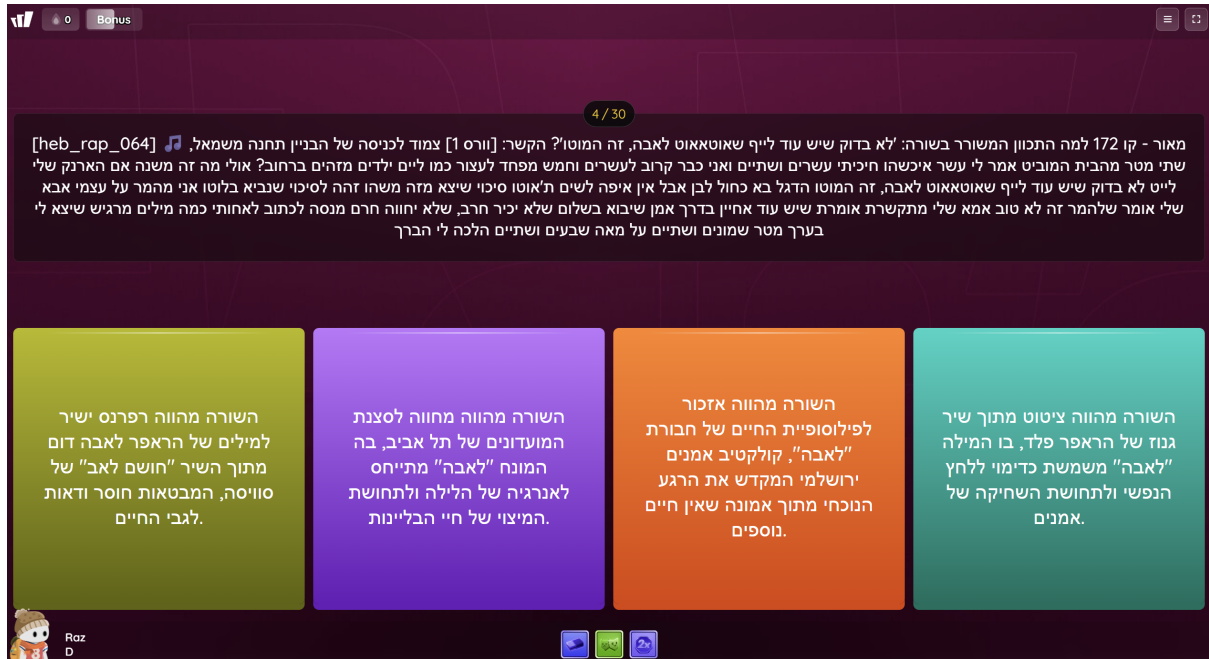


Figure 4: Screenshot from the Wayground Quiz interface

References

- [1] Avi Shmidman, Shaltiel Shmidman, and Moshe Koppel. DictaLM 2.0: A Hebrew Large Language Model. *arXiv preprint arXiv:2407.12345*, 2024.
- [2] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 46595–46623, 2023.